



Universidade do Minho

Escola de Engenharia

Programação Imperativa

LETI :: 2022/23

Novembro 2022

© António Esteves

© Slides adaptados de:

CSC230 - C e Software Tools, Computer Science Faculty, Duke University, USA

BE5B99CPL – C Programming Language, Jan Faigl, Czech Technical University in Prague

Bibliografia essencial:

C Programming: A Modern Approach, capítulo 8

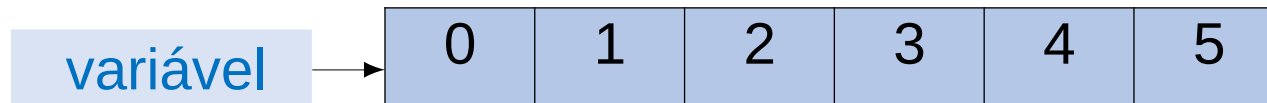
5. *Arrays* ou Vetores

- Declaração de *arrays*
- Armazenamento em memória e limites
- Operações sobre *arrays*
- *Arrays* de tamanho variável
- *Arrays* multidimensionais
- Operador `sizeof`

- **Declaração de *arrays***
- Armazenamento em memória e limites
- Operações sobre *arrays*
- *Arrays* de tamanho variável
- *Arrays* multidimensionais
- Operador `sizeof`

- Quase todos os programas que fazem algo relevante utilizam **ciclos** e **arrays**

- **array** = estrutura de dados que permite guardar **vários valores do mesmo tipo**



- O **nome** da variável representa o **endereço de memória** onde é guardado o **primeiro elemento** desse *array*
- O *array* é **declarado** como:
 - **tipo nome_array[numero_elementos] = {lista de valores} ;**
 - em que o número de elementos é uma **expressão constante**
- Para **aceder aos elementos** dum *array* utiliza-se o operador de subscrição de array **[]**: **nome_array [índice]**

- **a[n]** refere-se ao **n**-ésimo elemento do *array* **a**
 - o índice dos elementos começa em **0**
- O **nome** do *array* e o **índice** do elemento acedido são *comutativos*: **a[n]** refere-se ao **mesmo** valor que **n[a]**

```
dias_no_mes[0] = 31;
```

forma pouco utilizada → **1[dias_no_mes] = 28;**

- Um *array* é guardado num **bloco de memória contínua**
- Cuidado: A declaração de um *array* dentro duma função reserva memória da pilha (se não for definida como **static**) → e a pilha tem espaço bastante limitado!!!

- A **declaração** especifica
 1. o **tipo de dados** dos elementos
 2. o **número de elementos** do *array* (implícita ou explicitamente)
 3. os **valores iniciais** dos elementos, que podem ser parcialmente/totalmente/não especificados
- O tamanho do *array* pode ser qualquer expressão constante inteira
 - exemplos: **3** , **$5 * 32 - 24 / 4$** , ...

Declaração de *arrays* com uma dimensão (1)

➤ Tamanho **explícito** e **sem inicialização**:

```
int    dias_mes[12];  
char   letra_inicial[12];  
float  mm_chuva[12];
```

➤ Tamanho **explícito** e com **inicialização total**:

05-inic1-array.c

```
int dias_mes[12] =  
    {31,28,31,30,31,30,31,31,30,31,30,31};  
  
char letra_inicial[12] =  
    {'J','F','M','A','M','J','J','A','S','O','N','D'};  
  
float mm_chuva[12] =  
    {3.5,3.7,3.8,2.6,3.9,3.6,4.0,4.1,3.2,2.9,3.0,3.3};
```

O que acontece se tentarmos inicializar o *array* com mais de 12 valores?

➤ Tamanho **implícito** e com **inicialização total**:

```
int dias_mes[] =  
    {31,28,31,30,31,30,31,31,30,31,30,31};  
  
char letra_inicial[] =  
    {'J','F','M','A','M','J','J','A','S','O','N','D'};  
  
float mm_chuva[] =  
    {3.5,3.7,3.8,2.6,3.9,3.6,4.0,4.1,3.2,2.9,3.0,3.3};
```

05-inic2-array.c

O número de valores incluídos na inicialização determina o tamanho do *array*

Declaração de *arrays* de uma dimensão (3)

- Podemos inicializar **seletivamente** apenas alguns elementos

→ os elementos não especificados são inicializados com **zero**

05-inic3-array.c

```
int    dias_mes[12] = {31,28,31,30,31,30};  
char   letra_inicial[12] = {'J','F','M'};  
float  mm_chuva[12] = {3.5,3.7,3.8,2.6,3.9,3.6,4.0,4.1};
```

- Podemos ainda **alterar a ordem** dos elementos a inicializar

05-inic4-array.c

```
int    dias_mes[12]    = { [0]=31, [3]=30, [7]=31 };  
char   letra_inicial[12] = { [2]='M', [3]='A', [4]='M', [11]='D' };
```

- E será possível definir o tamanho do *array* **implicitamente** e utilizar inicialização **parcial**?

```
char letra_inicial[] =  
    { [0]='J', [2]='M', [8]='S' } ;
```

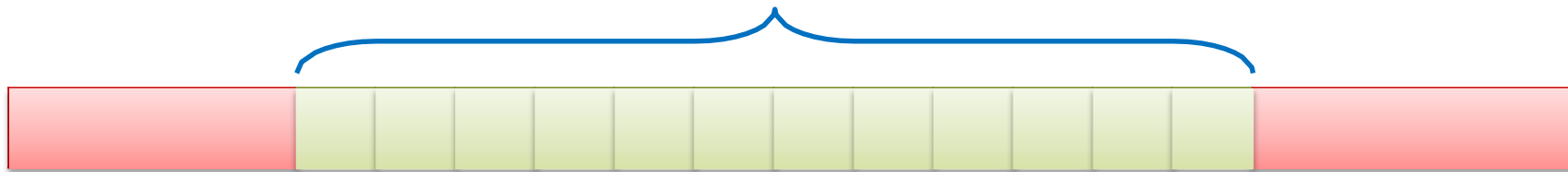
05-inic5-array.c

Qual o tamanho do *array*?

- Declaração de *arrays*
- **Armazenamento em memória e limites**
- Operações sobre *arrays*
- *Arrays* de tamanho variável
- *Arrays* multidimensionais
- Operador `sizeof`

armazenamento do *array*:

```
int dias_mes[12];
```



- Cada célula representada na figura guarda um **int**
- O compilador de C não verifica se o índice de acesso a um *array* é válido, ou seja, se tem um valor entre **0** e **tamanho_array-1**
- Assim podemos aceder à posição **76** ou **-35** do *array* **dias_mes** de tamanho **12**:
 - **dias_mes[76] !!!**
 - **dias_mes[-35] !!!**
- Claro que isto **deve ser evitado** pois não sabemos a que informação estamos a aceder

- Os acessos a posições fora dos limites de um *array* ...
 - podem gerar erros no programa, como é o caso do erro "**segmentation fault**", que obrigam o programador a terminar abruptamente e em falha
 - podem corromper indevidamente valores de outras variáveis
 - podem apenas ler valores errados
- Como em muitos programas as posições de acesso só são conhecidas durante a sua execução, este tipo de erro é um dos mais difíceis de diagnosticar em C

- Todos os elementos dum *array* ocupam um espaço em memória do **mesmo tamanho**
- Os elementos são acedidos através do **índice**/posição do elemento dentro do *array*
- Logo, a posição de memória em que é guardado cada elemento, relativamente ao início do *array*, é fixa:

$$\text{endereço relativo} = \text{tamanho_elemento} * \text{índice_elemento}$$

- E a posição de memória absoluta onde os valores são armazenados também é fixa:

$$\text{Endereço} = \text{endereço_1o_elemento} + \text{tamanho_elemento} * \text{índice_elemento}$$

- A memória para um *array* é reservada pela declaração da variável, a qual define o número de elementos (*= tamanho do array*) e o tamanho dos elementos (*= tamanho desse tipo de variável*)
- **O tamanho dum *array* declarado não pode ser mais alterado**

- Declaração de *arrays*
- Armazenamento em memória e limites
- **Operações sobre *arrays***
- *Arrays* de tamanho variável
- *Arrays* multidimensionais
- Operador `sizeof`

- As únicas **operações nativas** sobre *arrays* são:
 - aceder a um elemento do *array* através do operador de indexação **[]**
 - obter o endereço de um elemento com o operador **&**
 - obter o espaço ocupado em memória pelo *array* com o operador **sizeof()**
- Não há operadores para ...
 - atribuir o mesmo valor a todo os elementos do *array*
 - adicionar dois *arrays* (elemento a elemento)
 - multiplicar dois *arrays* (elemento a elemento)
 - ordenar o conteúdo do *array*
 - etc etc

- Em vez de se utilizar operadores nativos, temos que escrever **ciclos** para aplicar a mesma operação a cada elemento do *array*
- Um exemplo:

```
int exame1_notas[NUM_ALUNOS] ,  
    tpc1[NUM_ALUNOS] ,  
    tpc2[NUM_ALUNOS] ,  
    tpc[NUM_ALUNOS] ;  
  
for (int j = 0; j < NUM_ALUNOS ; j++) {  
    exame1_notas[j] = 100;  
    tpc[j] = tpc1[j] + tpc2[j];  
}
```

- Declaração de *arrays*
- Armazenamento em memória e limites
- Operações sobre *arrays*
- ***Arrays* de tamanho variável**
- *Arrays* multidimensionais
- Operador `sizeof`

- Em **C99**, o tamanho dos *arrays* (que não sejam qualificados como **static**) pode ser conhecido **durante a execução** do programa:

```
int meses;
```

05-array-tvariavel1.c

```
printf("Introduza #meses do projeto: ");  
scanf("%d", &meses);
```



```
int dias[meses];
```

O que acontece se o utilizador introduzir um valor zero ou negativo para tamanho do *array* **dias**?

- Embora o tamanho dum *array* possa ser definido dinamicamente, depois de definido **não pode ser mais alterado** durante a execução do programa

```
int num1, num2;  
printf("Insira o tamanho do array reg: ");  
scanf("%d", &num1);  
int reg[num1];  
  
// fazer algo com reg  
  
printf("Insira o novo tamanho de reg: ");  
scanf("%d", &num2);  
int reg[num2];
```

05-array-tvariavel2.c

Este código é inválido, resulta num erro ao compilar!

Arrays de tamanho variável: exemplo

```
#include <stdio.h>

int main(void)
{
    int i, n;
    printf("Indique a quantidade de inteiros a ler: ");
    scanf("%d", &n);

    int a[n]; /* array de tamanho variável */
    for (i = 0; i < n; ++i) {
        scanf("%d", &a[i]);
    }
    printf("Inteiros introduzidos por ordem inversa: ");
    for (i = n - 1; i >= 0; --i) {
        printf(" %d", a[i]);
    }
    printf("\n");
    return 0;
}
```

// 05-array-tvariavel3.c

- Declaração de *arrays*
- Armazenamento em memória e limites
- Operações sobre *arrays*
- *Arrays* de tamanho variável
- ***Arrays* multidimensionais**
- Operador `sizeof`

- **Declaração** de um *array* **multidimensional** com tamanho definido **explicitamente** (para todas as dimensões), **sem** inicialização:

```
int xy[10][20];
```

```
unsigned char pixel_rgb[256][256][3];
```

altura

largura

intensidade R, G, B

- **Indexar** um elemento do *array* multidimensional:

```
int xy_5_3 = xy[5][3];
```

```
unsigned char red = pixel_rgb[100][25][0];
```

- Na verdade um array multidimensional é *array de arrays*:
 - um *array* 2D (**xy**) é um vetor (*array* 1D) em que cada elemento deste vetor é um vetor (*array* 1D)
 - um *array* 3D (**pixel_rgb**) é um vetor (*array* 1D) de matrizes (*arrays* 2D), em que cada matriz é um *array* de *arrays* 1D
 - etc.
- As referências seguintes são todas válidas:

```
pixel_rgb           // valores R,G,B de todos os
                    // pixéis da imagem
pixel_rgb[9]        // todos os pixéis da 10ª linha
pixel_rgb[9][4]      // 5º pixel da 10ª linha
pixel_rgb[9][4][0]   // valor Red do 5º pixel da 10ª linha
```


Inicialização de *arrays* multidimensionais (1)



Universidade do Minho
Escola de Engenharia

- Inicialização **implícita**: percorrendo os valores a atribuir da esquerda para a direita, a regra de atribuição desses valores aos elementos do *array* pode resumir-se a: primeiro percorremos a dimensão mais à direita, depois a dimensão que se lhe segue, e assim sucessivamente até chegar à dimensão mais à esquerda
- Exemplo:

```
// array 2D com 2 linhas e 3 colunas
// 1º percorrem-se as colunas e depois as linhas
int m2D[2][3] =
    { {1, 2, 3}, {4, 5, 6} };

for (int i = 0; i < 2; i++)
    for (int j = 0; j < 3; j++)
        printf("%i ", m2D[i][j]);
```

- O programa escreve no terminal: **1 2 3 4 5 6**

05-array2d-1.c

➤ Inicialização **completa** e tamanho do *array* **explícito**:

```
int ck[3][4] =  
    { {0, 1, 2, 3}, // linha 0, colunas 0 a 4  
      {4, 5, 6, 7}, // linha 1, colunas 0 a 4  
      {8, 9, 10, 11} // linha 2, colunas 0 a 4  
    };
```

➤ Inicialização **parcial** e tamanho **explícito**:

```
int id[3][4] =  
    { {0, 1},  
      {4, 5},  
      {8, 9} };
```

Como fica a inicialização completa do *array* **id**?

Inicialização de *arrays* multidimensionais (3)



Universidade do Minho
Escola de Engenharia

- Se usarmos inicialização seletiva (indexada), os elementos não especificados vão ficar com valor **zero**

```
void escreve(int m[3][3])
{
    for (int l = 0; l < 3; ++l) {
        for (int c = 0; c < 3; ++c) {
            printf("%10i", m[l][c]);
        }
        printf("\n");
    }
}

int main(void) {
    int m0[3][3];
    int m1[3][3] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
    int m2[3][3] = {1, 2, 3, 4 };
    int m3[3][3] =
        { [0][0] = 1, [1][1] = 2, [2][2] = 3 };
    escreve(m0);
    escreve(m1);
    escreve(m2);
    escreve(m3);
}
```

m0: não inicializado

-5840	3276	4227
0	1	0
74014	0	0

m1: inic. por linhas

1	2	3
4	5	6
7	8	9

m2: inic. parcial

1	2	3
4	0	0
0	0	0

m3: inic. indexada

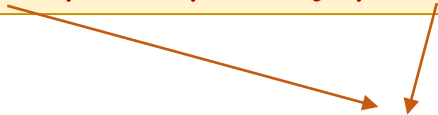
1	0	0
0	2	0
0	0	3

05-inic-array2d.c

- Apenas o tamanho da **dimensão mais à esquerda** pode ser omitido

OK

```
int ok[][3] =  
{ {0, 1, 2}, {4, 5, 6} };
```



Dado que a inicialização inclui 2 linhas de valores, o tamanho da primeira dimensão do *array* será **2**.

Inválido!

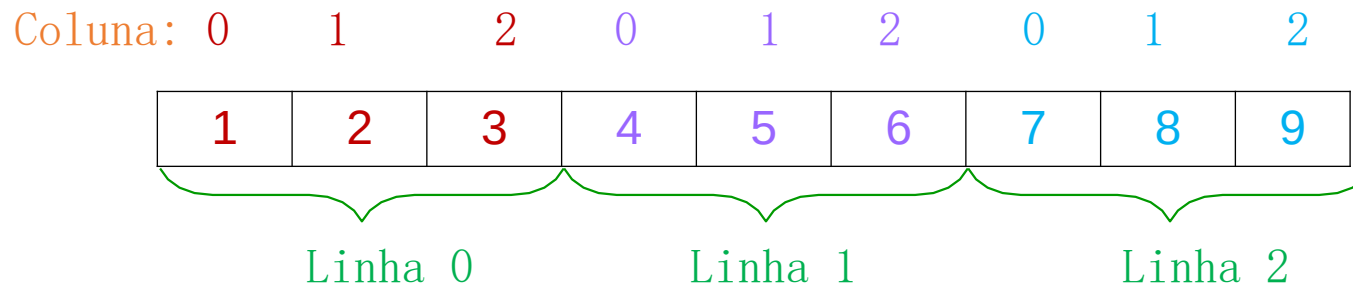
```
int nok[2][] =  
{ {0, 1, 2}, {4, 5, 6} };
```

Armazenamento de *arrays* multidimensionais em memória

- Um *array* multidimensional também ocupa **sempre** um bloco contínuo em memória
- Por exemplo, à declaração `int m[3][3];` corresponde um espaço em memória com tamanho `3*3*sizeof(int)`, ou seja, geralmente corresponde a **36** bytes

```
int m[3][3] = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };  
for (int l = 0; l < 3; ++l) {  
    for (int c = 0; c < 3; ++c) {  
        printf("%p ", &m[l][c]);  
    }  
    printf("\n");  
}
```

Armazenamento dos elementos segue a ordenação **row major**



- Declaração de *arrays*
- Armazenamento em memória e limites
- Operações sobre *arrays*
- *Arrays* de tamanho variável
- *Arrays* multidimensionais
- Operador **sizeof**

O operador `sizeof()` (1)

- Não é uma chamada de função, é um operador do C
 - devolve o **número de bytes que ocupa** um tipo de dados ou uma variável (de um certo tipo)
- O valor devolvido é do tipo `size_t` ⇔ `unsigned long`

```
#include <stdlib.h>
size_t t1, t2, t3;
int     a;
float   b[100];
```

```
t1 = sizeof(a);
t2 = sizeof(b);
t3 = sizeof(b[0]);
```

} Quais os
valores de
t1, t2 e t3?

05-array-sizeof.c

- Também pode ser utilizado para obter o número de elementos num *array*

```
float b[100];  
...  
int nElems;  
nElems = sizeof(b) / sizeof(b[0]);
```

$\text{\#elementos} = \text{tamanho_array} / \text{tamanho_elemento}$

Exercício 1: números pares numa sequência de números

- Utilizando um *array* e ciclos, escreva um programa que pede ao utilizador para introduzir 10 números inteiros e depois escreve no terminal apenas os números pares previamente introduzidos.
- Exemplo de utilização do programa:

```
./pares10  
2 3 6 7 12 13 17 18 20 29  
2 6 12 18 20
```